

Agora

Konuların tartışılıp oylamayla karara bağlandığı forum — proje raporu

Abdülbaki Bayır

24290050

YZM327 · YAZILIM MÜHENDİSLİĞİNE GİRİŞ

Agora — Proje Raporu

Python 3.11+, Flask 3.1, SQLite · Canlı sürüm (kurulum gerektirmez): baki-b.github.io/YZM327-Odevler · Kaynak kod: github.com/Baki-B/YZM327-Odevler

Parantez içindeki atıflar ders sunumlarındaki slaytları gösteriyor: TD = Yazılım Tasarım Desenleri, H2 = Problem Çerçeveleme, S01 = YZ Mühendisliğine Giriş, S02 = Temel Modelleri Anlamak. Örneğin TD-43, Tasarım Desenleri sunumunun 43. slaytı.

İçindekiler

1. Proje özeti ve ödev gereksinimleri
2. Kullanıcılar ve kullanım durumları
3. Karar süreci ve oylama kuralları
4. Mimari
5. Veri modeli ve tutarlılık
6. Temel akışlar
7. Yönetmelik denetimi
8. Kayıt defteri ve oy makbuzu
9. Yapay zeka üye
10. SOLID ilkeleri ve tasarım desenleri
11. Problem çerçeveleme ve ölçüm
12. Güvenlik ve kişisel veri
13. Test, kalite ve başarım
14. Dağıtım ve mobil
15. Mini laboratuvar
16. Sınırlar ve YZ kullanım beyanı

1. Proje özeti ve ödev gereksinimleri

Agora, bir topluluğun bir konuyu tartışıp oylamayla tek bir karara bağlaması için tasarlanmış bir forum. Süreç şöyle işliyor: Bir üye konu açtığında 24 saatlik tartışma dönemi başlıyor. Bu dönemde herkes mesaj yazabiliyor, ama her katılımcı en fazla **bir fikir** önerebiliyor. Tartışma bitince en fazla beş tur süren oylamaya geçiliyor ve her turda az oy alan fikirler eleniyor. Bir fikir %75 oya ulaşırsa ya da yarışta tek fikir kalırsa konu hemen karara bağlanıyor. Beşinci turun sonunda ise en çok oyu alan fikir kazanıyor.

Forumda mesajlar silinemiyor; bir mesajı gizlemek için bile oylama yapılması gerekiyor. Konu açma, oy verme ve sonuç gibi önemli olayların hepsi, sonradan değiştirilemeyen bir kayıt defterine yazılıyor. Aşağıdaki tablo, ödevde istenen her maddenin Agora'da nasıl karşılandığını gösteriyor.

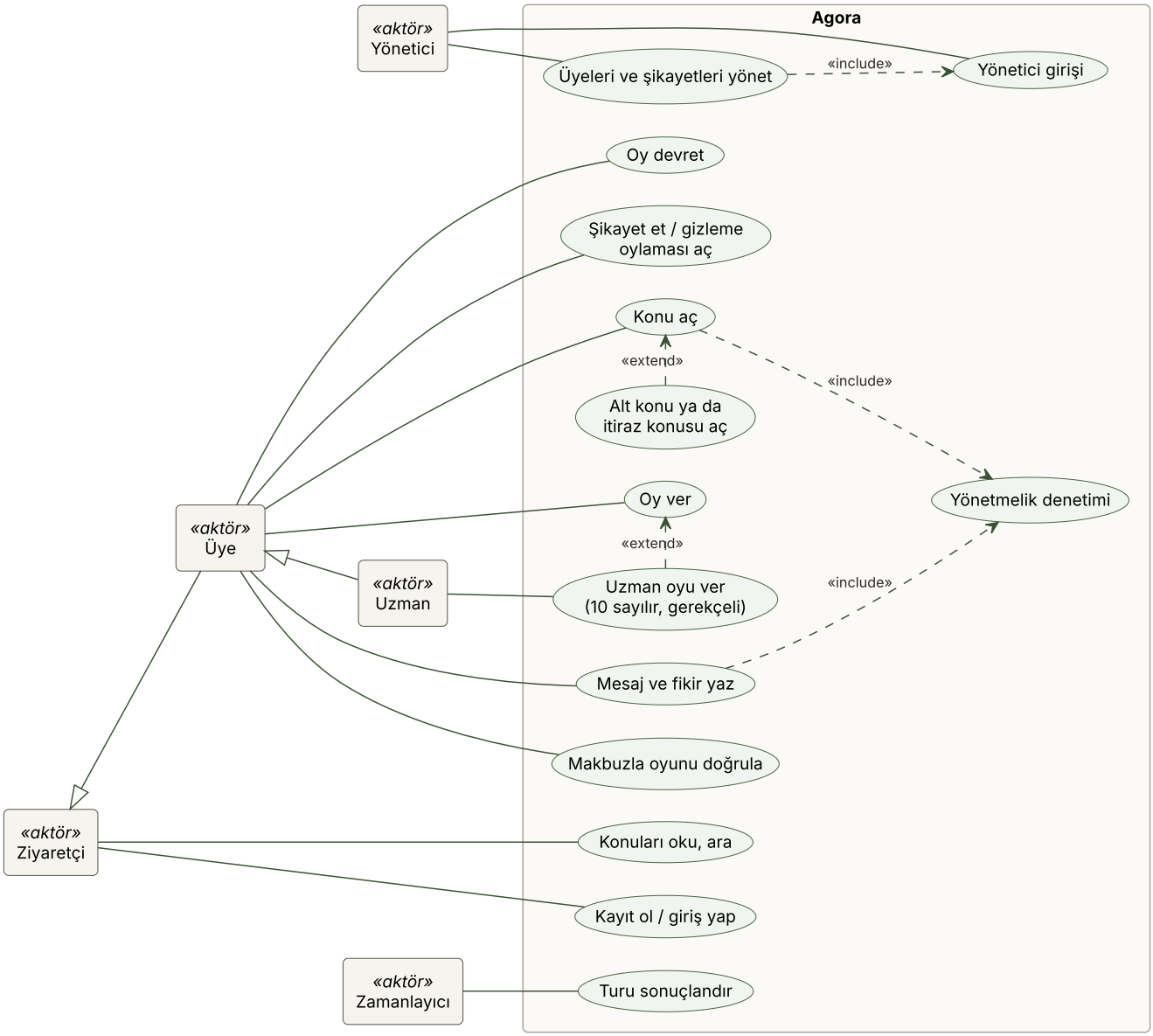
Ödevde istenen	Agora'daki karşılığı
Herkes konu açabilmeli, düzenleyebilmeli, silinmesini önerebilmeli; alt konular olmalı	Konu onay beklemeden hemen açılıyor. Tartışma sürerken sahibi konuyu düzenleyebiliyor ve eski hâli saklanıyor. Bir konuyu kaldırmak için 3/4 oy gerekiyor. Alt konu, üst konunun katılım kuralını yalnızca daraltabiliyor.
Kararlar çoğunlukla alınmalı; konu sonunda fikirler uzlaşmalı	Turlarla eleme, %75 ezici üstünlük ve yeter sayı kuralı var. Süreç sonunda tek bir fikir karar oluyor. Karara katılmayan üye itiraz konusu açabiliyor.
Giriş ekranı olmalı; kayıtta ad soyad, adres, doğum tarihi, şifre ve takma ad istenmeli	Kayıt ve giriş sayfaları var. Ad, adres ve doğum tarihi gizli tutuluyor; diğer üyeler yalnızca takma adı görüyor. Şifresini unutan üye kurtarma koduyla yeni şifre belirleyebiliyor.
Kişi bilgilerine göre kısıt konabilmeli (ör. İstanbul konusuna yalnızca İstanbullular katılsın)	Konuya konum (il › ilçe) ve yaş kuralı eklenebiliyor. Kuralı sağlamayan üye o konuda *gözlemci* oluyor: konuyu okuyabiliyor ama oy veremiyor.
Herkes eşit olmalı, ama bazıları daha eşit olabilmeli	Kurallar herkes için aynı. Uzmanın oyu kendi alanında 10 sayılıyor; ancak "çift çoğunluk" kuralı, birkaç uzmanın kalabalığı tek başına yenmesini engelliyor.

Ödevde istenen	Agora'daki karşılığı
Kurallar ve uygunluk bir ontolojiyle ölçülmeli; yönetmelik denetlemeli	Uygunluk; konum ağacı (Türkiye › bölge › il › ilçe), kategori ağacı ve kategori kavramlarından oluşan bir ontolojiyle hesaplanıyor. Yönetmeliğin D1–D7 maddeleri konuları ve mesajları otomatik olarak denetliyor. Yönetmeliğin kendisi de oylamayla değişebiliyor.
Tartışmalar silinmemeli; silinecekse oylanmalı	Mesajlar silinemiyor. Gizleme 3/4 oylamayla yapılıyor ve iz bırakıyor. Düzenlenen mesajların eski hâli de görülebiliyor.
Oy hakkı devredilebilmeli; azınlık korunmalı	Oy hakkı genel, kategori ya da konu kapsamında devredilebiliyor ve bir kişi en fazla 5 devir taşıyabiliyor. Azınlığı itiraz konusu, 3/4 eşikler, korunan temel haklar ve gizli oy koruyor.
Hem uygulama hem web sitesi olmalı	Web sitesi, telefona kurulabilen web uygulaması (PWA), Android uygulaması (APK), Expo Go ve REST API.
Dağıtık defter, bilirkişi ve yapay zeka kullanılmalı; kişiler bir graf üzerinde gösterilmeli	Üç kopyalı kayıt defteri ve oy makbuzu; uzmanlık oylaması; özet yazan yapay zeka üye; üye ağı (PageRank ile etki, Gini katsayısıyla eşitsizlik ölçümü).
Ders konuları: problem çerçeveleme, SOLID ve tasarım desenleri, UML	Problem çerçeveleme Bölüm 11'de, SOLID ve desenler Bölüm 10'da, UML diyagramları Şekil 1–11'de.

2. Kullanıcılar ve kullanım durumları

Sistemde altı tür aktör var. Bunlardan biri, zamanlayıcı, bir kişi değil; sistemin kendi içinde çalışan bir görev.

Aktör	Yapabildikleri
Ziyaretçi	Konuları ve kayıt defterini okuma, arama yapma, kayıt olma ve giriş yapma.
Üye	Ziyaretçinin yapabildiklerine ek olarak konu açma, mesaj ve fikir yazma, oy verme, oyunu makbuzla doğrulama, oy devretme, şikayet etme ve uzmanlığa başvurma.
Gözlemci	Ayrı bir hesap türü değil. Bir konunun konum ya da yaş kuralını sağlamayan üye o konuda gözlemci sayılıyor: konuyu okuyabiliyor, ama oy veremiyor ve fikir yazamıyor.
Uzman	Alanındaki üyelerin oylamasıyla seçilmiş üye. Kendi alanındaki fikir oylamalarında oyu 10 sayılıyor. Buna karşılık oyu herkese açık ve oyuna gerekçe yazması gerekiyor.
Yönetici	Üye girişinden ayrı bir yönetici girişiyle panele giriyor. Üyeleri askıya alma, şikayetleri oylamaya taşıma, site duyurusu yayımlama ve yedek alma gibi işleri yapıyor. Oylamaların sonucunu değiştiremiyor ve içerik silemiyor.
Zamanlayıcı (sistem)	Süresi dolan tartışmaları oylamaya geçiriyor ve süresi biten turları sonuçlandırıyor.



Şekil 1. Kullanım durumu diyagramı. Konu açarken ve mesaj yazarken yönetmelik denetimi her seferinde çalışıyor («include»). Alt konu ve itiraz konusu açmak, konu açmanın özel birer hâli («extend»). Uzman, üyenin özelleşmiş bir türü; bu yüzden üyenin yapabildiği her şeyi yapabiliyor.

3. Karar süreci ve oylama kuralları

Bir konu açıldığında önce tartışma, ardından oylama turları başlıyor. Konu sonunda ya bir fikri kabul ederek karara bağlanıyor, ya sonuçsuz kalıyor ya da oylamayla kaldırılıyor. Turların kuralları bu bölümde anlatılıyor; konunun geçebileceği bütün durumlar ise Şekil 2'de özetleniyor.

3.1 Turlar ve eleme

Her aşamanın süresi ve eleme eşiği şöyle:

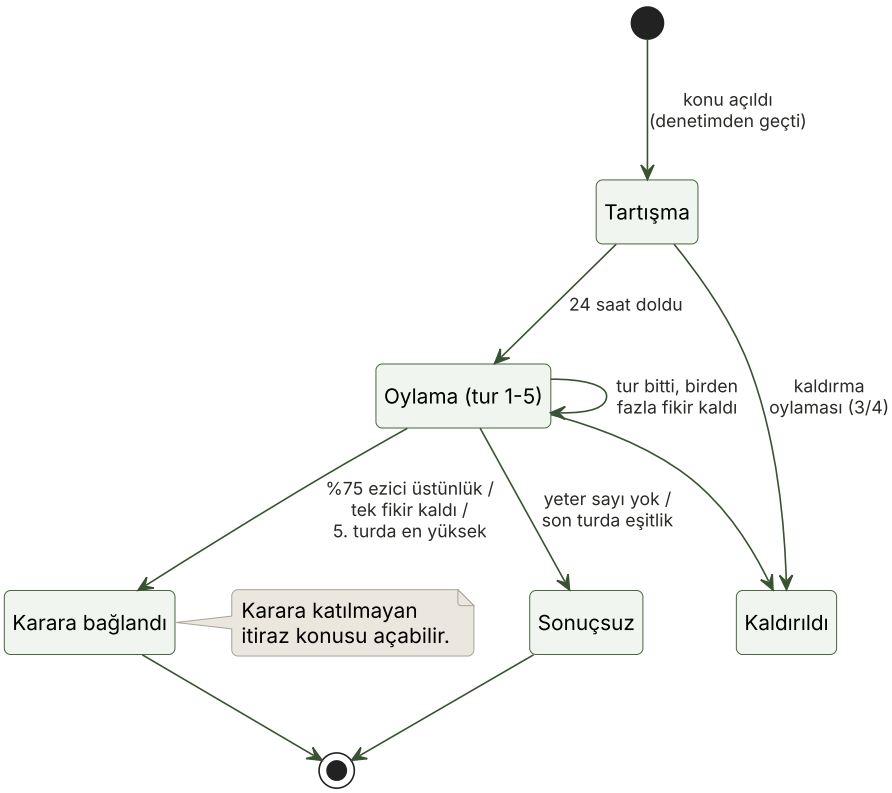
Aşama	Süre	Kural
Tartışma	24 saat	Mesajlar ve fikirler yazılıyor; her katılımcının bir fikir hakkı var.
1. tur	48 saat	%5'in altında kalan fikirler eleniyor. Bu turda yeni fikir de eklenebiliyor.
2. tur	24 saat	%5'in altında kalan fikirler eleniyor.

Aşama	Süre	Kural
3. tur	24 saat	%10'un altında kalan fikirler eleniyor.
4. tur	24 saat	%20'nin altında kalan fikirler eleniyor.
5. tur	24 saat	En çok oyu alan fikir kabul ediliyor. Oylar eşitse konu sonuçsuz kalıyor.

Turların erken bitmesini ya da geçersiz sayılmasını sağlayan üç kural daha var:

- **Ezici üstünlük:** Bir fikir herhangi bir turda %75 ya da daha fazla oy alırsa, sonraki turlar beklenmeden kabul ediliyor.
- **Tek kalan:** Eleme sonunda yalnızca bir fikir kaldıysa o fikir kabul ediliyor.
- **Yeter sayı:** Oy hakkı olanların en az %20'si (ve en az 2 kişi) oy kullanmazsa oylama geçersiz sayılıyor ve konu sonuçsuz kapanıyor.

> **Oran nasıl hesaplanıyor? (çift çoğunluk)** Bir fikrin oranı iki ayrı yoldan hesaplanıyor: *ağırlıklı oylara göre* (uzman oyu 10 sayılıyor) ve *kişi sayısına göre* (herkesin oyu 1 sayılıyor). Geçerli oran, bu ikisinden **küçük olanı**. Çekimser oylar da paydaya ekleniyor; böylece katılımcıların çoğu çekimser kaldığında bir fikir düşük destekle %75'e ulaşamıyor.
>
> *Örnek:* 10 kişilik bir oylamada 2 uzman A fikrine, 8 üye de B fikrine oy veriyor. Ağırlıklı oyların toplamı 2 × 10 + 8 = 28. A fikri ağırlıklı oylarda 20/28 = %71, kişi sayısında 2/10 = %20 alıyor; geçerli oranı küçük olan, yani %20. B fikri ağırlıklı oylarda 8/28 = %29, kişi sayısında 8/10 = %80 alıyor; geçerli oranı %29. Bu sayede birkaç uzman kalabalığı tek başına yenemiyor, kalabalık da uzmanları yok sayamıyor.



Şekil 2. Konunun durum diyagramı. Her durum, kendi içinde hangi işlemlere izin verildiğini biliyor. Kodda bu yapı State deseniyle kuruldu (Bölüm 10).

3.2 Diğer oylamalar

Fikir oylaması dışındaki oylamalar evet/hayır biçiminde yapılıyor. Her birinin eşiği ve süresi farklı:

Oylama	Eşik	Süre
Mesaj gizleme, konu kaldırma	3/4	48 saat
Uzmanlık başvurusu (alanın üyeleri oy veriyor)	2/3	48 saat
Yeni kategori (bütün üyeler oy veriyor)	Salt çoğunluk	48 saat
Yönetmelik değişikliği	2/3	96 saat
Temel hak maddesi ya da korunan parametre	3/4	96 saat

Bu sayıların hiçbiri kodda sabit değil; hepsi yönetmeliğin **parametresi** olarak tutuluyor. Topluluk bir parametreyi oylamayla değiştirebiliyor. Yine de her parametrenin anlamlı bir aralığı var: örneğin uzman ağırlığı 0 yapılamıyor, çünkü o zaman uzmanların oyu hiç sayılmazdı.

3.3 Katılım, uzmanlık ve oy devri

- **Katılım kuralı:** Konuyu açan üye, konuya bir konum (il ya da ilçe) ve yaş aralığı ekleyebiliyor. Konum kuralı konum ağacına göre değerlendiriliyor; örneğin "İstanbul" kuralını Kadıköy'de oturan üye de sağlıyor. Adres değişikliği ancak 30 gün sonra geçerli oluyor. Böylece kimse oylamadan hemen önce adresini değiştirip başka bir şehrin kararına katılamıyor.
- **Uzmanlık:** Uzman olmak için üç şart aranıyor. Aday o alanda en az 5 mesaj yazmış ve en az 2 farklı konuya katılmış olmalı. Kontenjan dolmamış olmalı (her ana kategoride en fazla 3 uzman). Son olarak alanın üyelerinden 2/3 onay almalı. Uzmanlık 180 gün geçerli. Yönetici kimseyi uzman atayamıyor; uzmanlığı yalnızca topluluğun oyu veriyor.
- **Oy devri:** Üye, oy hakkını genel olarak, bir kategori için ya da tek bir konu için başka bir üyeye devredebiliyor. Birden fazla devir varsa en dar kapsamlı olan geçerli oluyor: konu devri kategori devrinin, kategori devri de genel devrin önüne geçiyor. Bir kişi en fazla 5 devredilmiş oy taşıyabiliyor; bu sınır oyların birkaç kişide toplanmasını önlüyor. Üye bir oylamada kendisi oy verirse, o oylamadaki devri kendiliğinden geçersiz oluyor.

3.4 Azınlığı koruyan kurallar

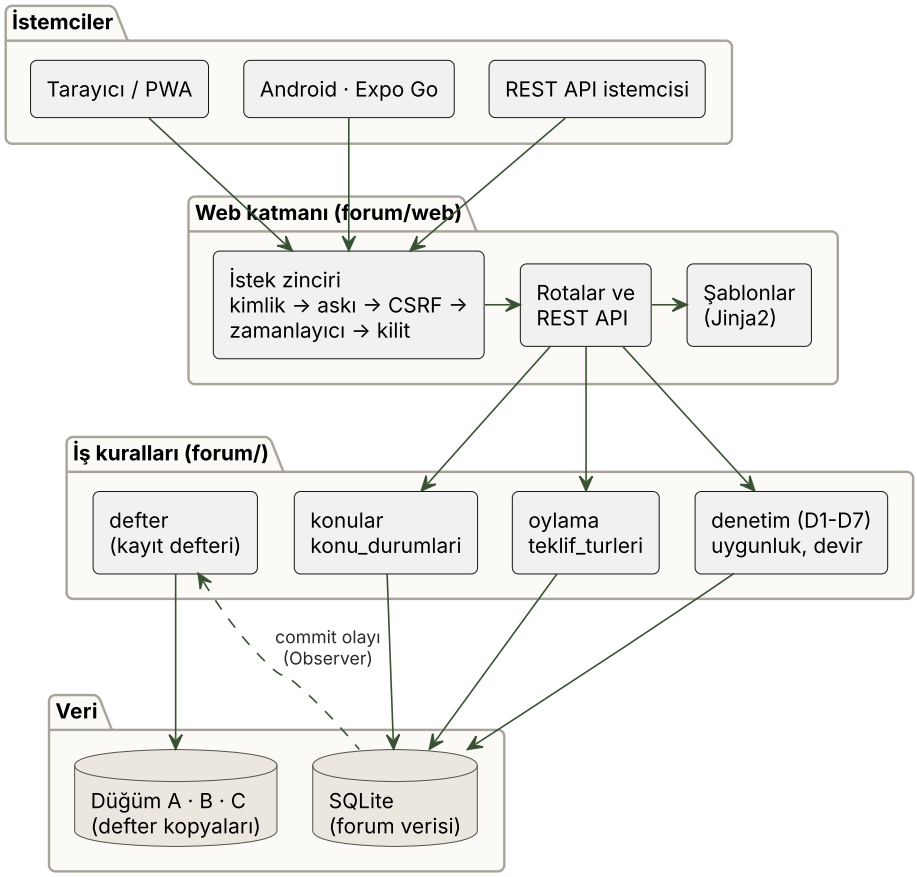
Çoğunluk kararının azınlığı ezmeye dönüşmemesi için şu kurallar kondu:

- Herkes fikir yazabiliyor; hangi fikirlerin oylamaya gireceğine çoğunluk karar vermiyor.
- Çift çoğunluk kuralı sayesinde ne uzmanlar ne de kalabalık tek başına karar verebiliyor.
- Çekimser oylar paydada sayıldığı için az sayıda destekle karar çıkmıyor.
- Mesaj gizleme ve konu kaldırma için 3/4 oy gerekiyor; salt çoğunluk bir azınlığın sesini kısamıyor.
- Temel hak maddeleri ancak 3/4 oyla değiştirilebiliyor.
- Devir tavanı (5) ve uzman kontenjanı (3), gücün birkaç kişide toplanmasını engelliyor.
- Üyelerin oyları gizli tutuluyor; kimse verdiği oy yüzünden baskı görmüyor. Yalnızca uzman oyları açık.
- Sonuca katılmayan üye itiraz konusu açarak kararı yeniden tartışmaya açabiliyor.

4. Mimari

Uygulama üç katmandan oluşuyor (Şekil 3). Bağımlılıklar yalnızca yukarıdan aşağıya doğru: üstteki katman alttakini kullanıyor, alttaki katman üsttekini tanımıyor.

- **Web katmanı** (forum/web klasörü, 13 modül): HTTP isteklerini karşılıyor, sayfaları şablonlardan üretiyor ve REST API sunuyor. İş kuralı içermiyor; isteği ilgili iş kuralı modülüne iletiyor.
- **İş kuralları** (forum klasörü, 35 modül): Konu, oylama, denetim, uygunluk ve defter gibi bütün kurallar burada. Bu modüller web katmanından bağımsız olduğu için tarayıcı açmadan, doğrudan test edilebiliyor.
- **Veri:** SQLite veritabanı (23 tablo ve bir tam metin arama dizini) ile kayıt defterinin üç ayrı kopyası.



Şekil 3. Katmanlar. Veritabanı kayıt defterini tanımıyor; defter, veritabanının "işlem kaydedildi" (commit) olayına abone oluyor. Kesikli ok bu aboneliği gösteriyor.

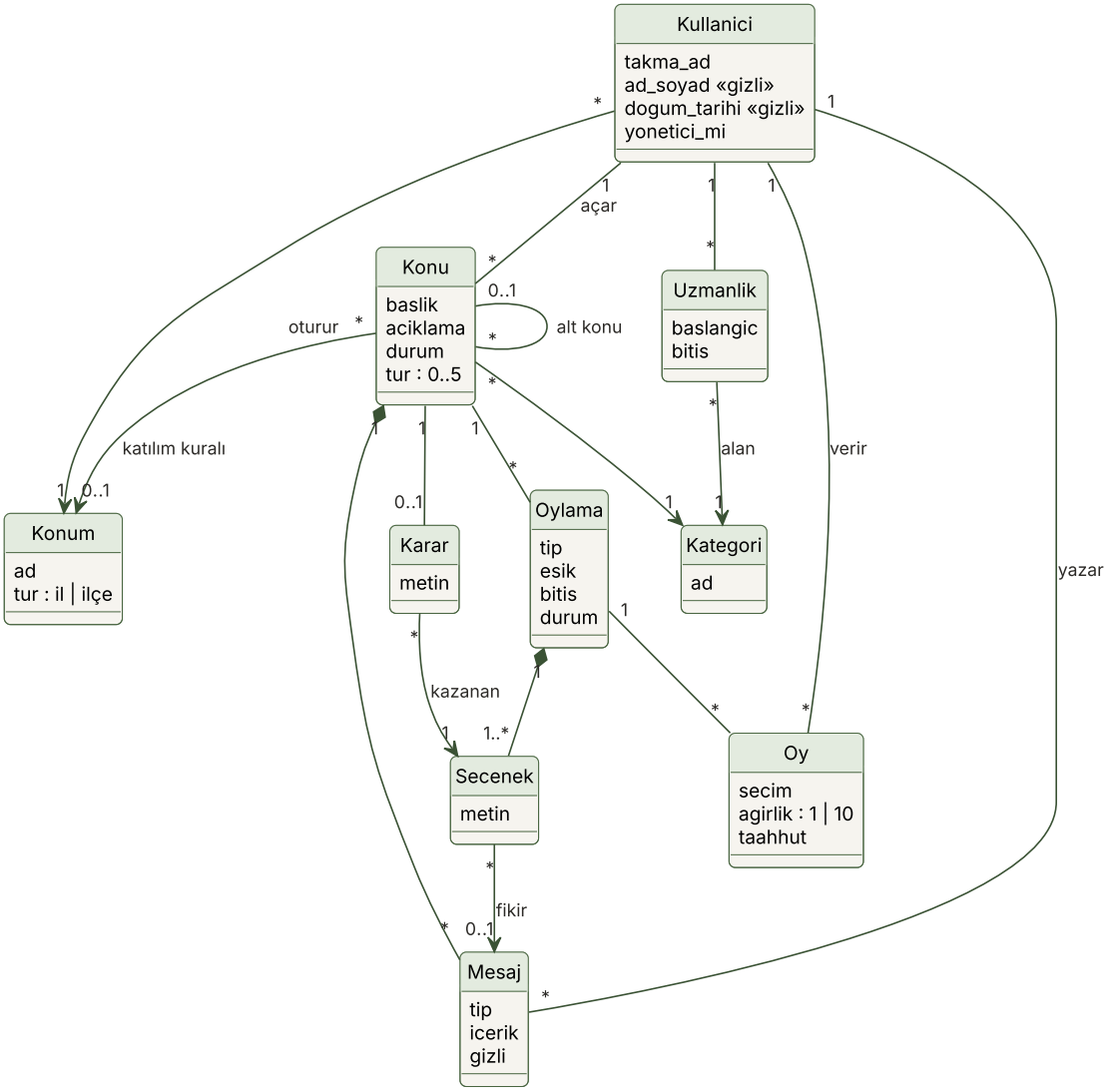
Her istek, rotaya ulaşmadan önce aynı zincirden geçiyor: **kimlik → askı → CSRF → zamanlayıcı → yazma kilidi**. Bu adımlar sırasıyla giriş yapan üyeyi buluyor, askıya alınmış üyeyi durduruyor, formun gerçekten bu siteden geldiğini doğruluyor, süresi dolan oylamaları sonuçlandırıyor ve aynı anda iki yazma işlemini önlüyor. Zincirdeki bir adım isteği kendisi yanıtlarsa (örneğin CSRF anahtarı yanlışsa hata sayfası dönerse) sonraki adımlar çalışmıyor.

5. Veri modeli ve tutarlılık

Veritabanındaki ana kavramlar ve aralarındaki ilişkiler Şekil 4'te görülüyor. Modelle ilgili önemli noktalar:

- Bir konunun birden çok oylaması olabiliyor. Her fikir turu ayrı bir oylama sayılıyor; mesaj gizleme ve konu kaldırma oylamaları da konuya bağlı.

- Ad soyad ve doğum tarihi gizli alanlar. Bu bilgiler yalnızca katılım kuralını hesaplamak için kullanılıyor; hiçbir sayfada ve API yanıtında görünmüyor.
- Konuların ve mesajların düzenlenmeden önceki hâlleri ayrı tablolarda saklanıyor. Böylece okuyan herkes neyin değiştiğini görebiliyor.
- Oy kaydında seçimin yanında bir **taahhüt** (SHA-256 özeti) de tutuluyor. Deftere seçimin kendisi değil, bu özet yazılıyor (Bölüm 8).



Şekil 4. Alan modeli (sınıf diyagramı). Diyagramdaki "Oylama" kavramının koddaki adı *teklif*. "Seçenek" ise bir fikir ya da evet/hayır seçeneklerinden biri olabiliyor.

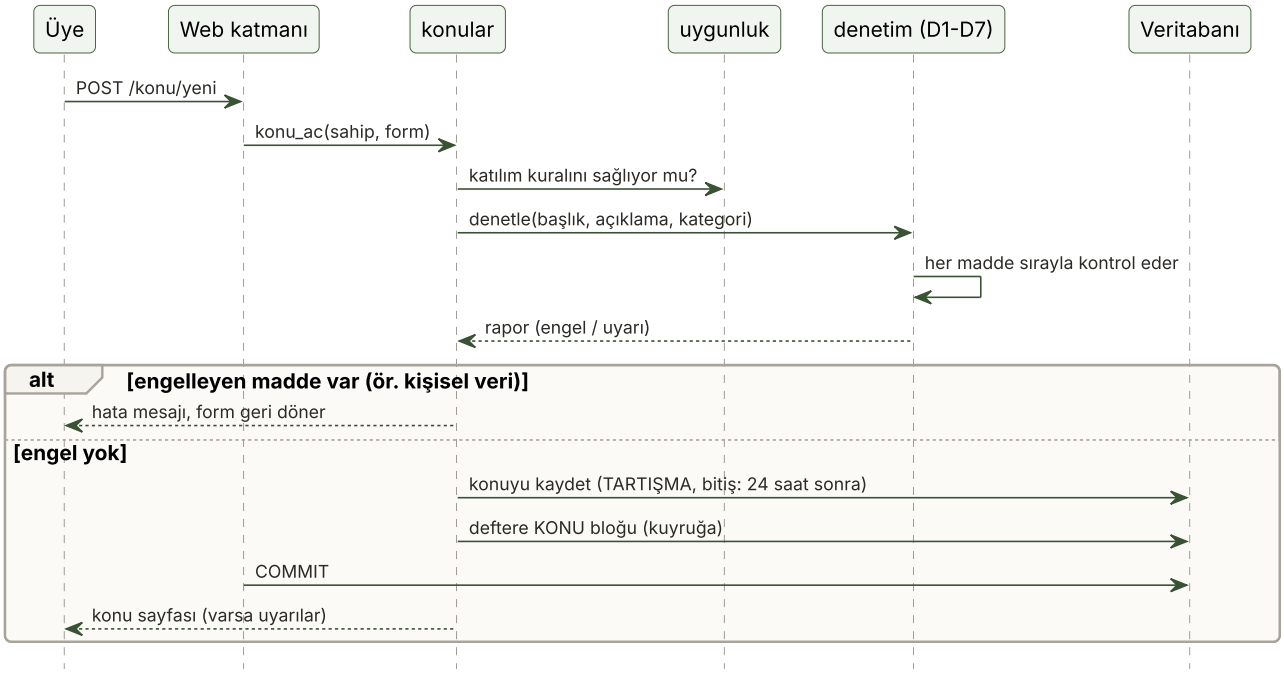
Aynı anda gelen isteklerde tutarlılık. İki üye aynı anda oy verdiğinde ya da bir oy tam oylama kapanırken geldiğinde verinin bozulmaması gerekiyor. Bunun için dört önlem alındı:

- Yazma yapan her istek, işe başlarken `BEGIN IMMEDIATE` komutuyla veritabanı kilidini alıyor. Böylece iki yazma işlemi birbirinin arasına giremiyor.
- Oy, yalnızca oylama hâlâ açıksa kaydediliyor (koşullu `INSERT`). Kişi başına tek fikir kuralını da veritabanındaki tekil indeks koruyor. Bu davranışı, 8 iş parçacığının aynı anda yazdığı bir test denetliyor.
- Zamanlayıcı her oylamayı ayrı bir kayıt noktasında (`SAVEPOINT`) işliyor. Bir oylamada hata çıkarsa yalnızca o oylamanın değişiklikleri geri alınıyor; diğer oylamalar etkilenmiyor.
- Defter bloğu ve bildirim gibi yan etkiler önce kuyruğa alınıyor ve ancak işlem başarıyla kaydedildikten sonra uygulanıyor. Böylece geri alınan bir işlem deftere yanlış kayıt bırakmıyor.

6. Temel akışlar

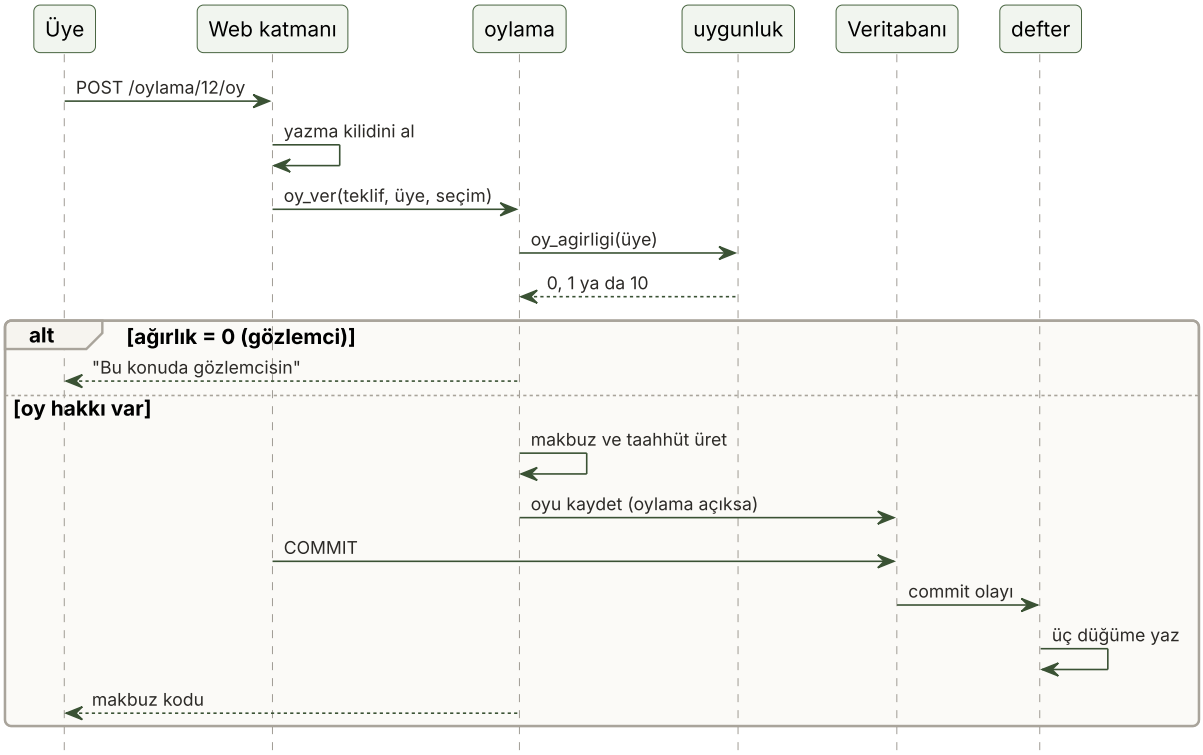
Bu bölümdeki sıralı diyagramlar, dört temel işlemde hangi modülün hangi sırayla çalıştığını gösteriyor.

6.1 Konu açma



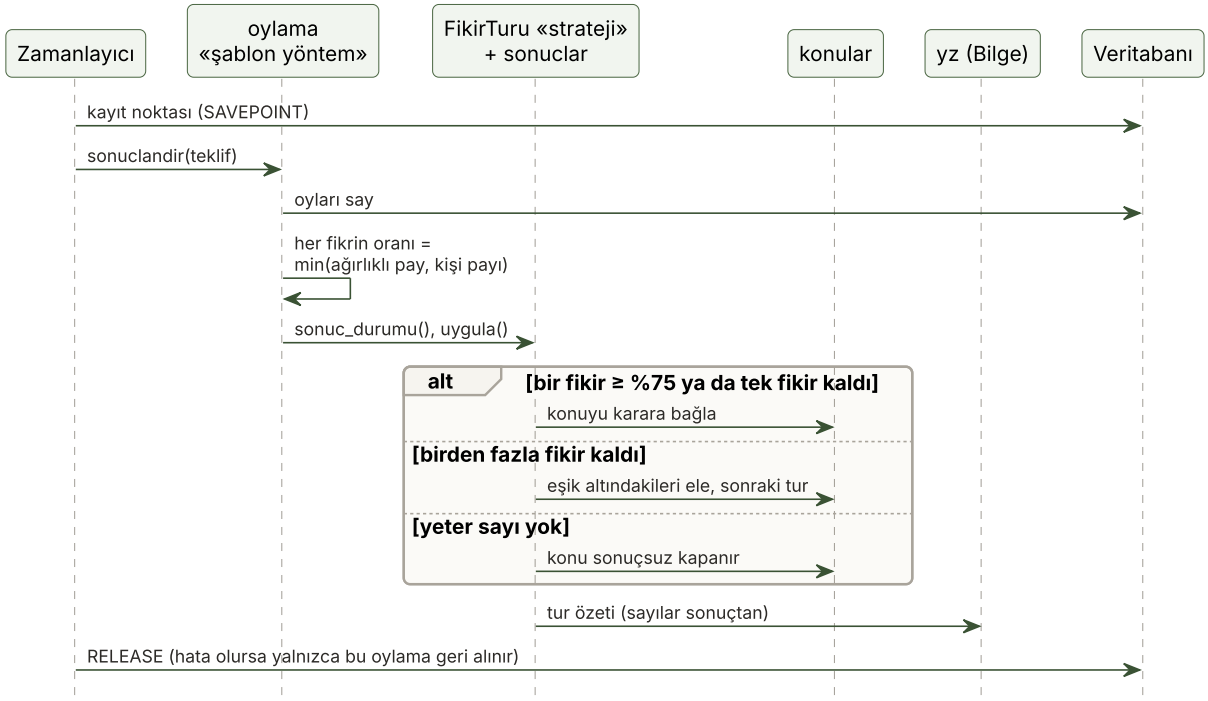
Şekil 5. Konu açma. Konu kaydedilmeden önce yönetmelik denetiminden geçiyor. Engelleyen bir madde varsa (örneğin metinde telefon numarası geçiyorsa) konu açılmıyor ve form bir hata mesajıyla geri dönüyor.

6.2 Oy verme



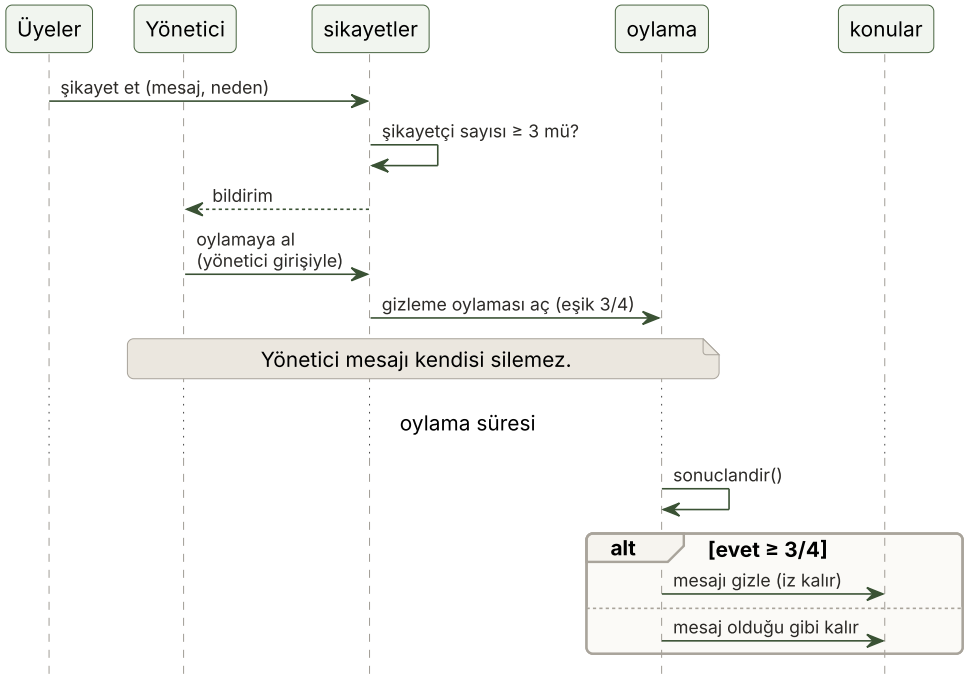
Şekil 6. Oy verme. Oyun ağırlığını uygunluk modülü belirliyor: gözlemci için 0, üye için 1, uzman için 10. Oy kaydedilip işlem tamamlanınca veritabanı defteri haberdar ediyor ve taahhüt üç düğüme yazılıyor. Üye, makbuz kodunu yalnızca bir kez görüyor.

6.3 Tur sonu



Şekil 7. Tur sonu. Sonuçlandırma adımları her oylama türünde aynı sırayla çalışıyor (Template Method). Sonucun nasıl yorumlanacağı ise oylama türüne göre değişiyor (Strategy). Turun kararını veren işlev veritabanına dokunmuyor; bu yüzden ayrıca test edilebiliyor.

6.4 Şikayetten gizleme oylamasına



Şekil 8. Aynı içeriği en az 3 üye şikayet edince yöneticiye bildirim gidiyor. Yönetici içeriği kendisi silemiyor; yalnızca gizleme oylaması başlatabiliyor. Kararı 3/4 eşikli oylama veriyor.

7. Yönetmelik denetimi

Yeni bir konu ya da mesaj, kaydedilmeden önce yönetmeliğin denetim maddelerinden sırayla geçiyor. Her maddenin bir **ciddiyet düzeyi** var. *Engeller* düzeyindeki madde metnin gönderilmesini durduruyor. *Uyarır* düzeyindeki madde metni yayımlıyor, ama yazana bir uyarı gösteriyor. *Kapalı* bir madde ise hiç çalışmıyor. Bu düzeyler de oylamayla değiştirilebiliyor.

Madde	Ad	Neyi kontrol ediyor	Varsayılan
D1	Saygılı dil	Hakaret listesindeki kelimeler (Türkçe çekim ekleriyle birlikte)	Uyarır
D2	Kişisel veri	Telefon, e-posta, geçerli T.C. kimlik numarası, IBAN	Engeller
D3	Kategoriye uygunluk	Metin, seçilen kategorinin kavramlarını içeriyor mu? İçermiyorsa daha uygun bir kategori öneriliyor.	Uyarır
D4	Konum tutarlılığı	Metinde bir yer adı geçiyorsa, katılım kuralı o yerle uyumlu mu?	Uyarır
D5	Benzer konu	Benzer başlıklı açık bir konu var mı? Varsa üye o konuya yönlendiriliyor.	Uyarır
D6	Alt konu ilişkisi	Alt konu, üst konusuyla aynı kategoride mi?	Uyarır
D7	Açıklık	Açıklama, derdini anlatacak uzunlukta mı? (en az 40 karakter)	Uyarır

D1 başlangıçta "Engeller" düzeyindeydi. Ölçümde masum mesajları da engellediği görülünce "Uyarır" düzeyine indirildi (Bölüm 11). Kodda her madde bir zincirin halkası olarak yazıldı (Chain of Responsibility, Bölüm 10). Bu sayede yeni bir madde eklemek için mevcut kodu değiştirmek gerekmiyor; yeni bir sınıf yazıp zincire eklemek yetiyor.

8. Kayıt defteri ve oy makbuzu

Oylamaların sonradan değiştirilmediğini kanıtlamak için her önemli olay bir kayıt defterine yazılıyor. Defterin dört temel özelliği var:

- **Hash zinciri:** Her olay (konunun açılması, oy verilmesi, sonucun çıkması gibi) bir bloğa yazılıyor ve her blok bir önceki bloğun SHA-256 özetini taşıyor. Eski bir bloktaki tek bir harf bile değişirse sonraki bütün özetler tutmaz hâle geliyor; böylece değişiklik hemen fark ediliyor.
- **Üç kopya:** Zincir üç ayrı düğümde (A, B, C) tutuluyor. Düğümler uyuşmazsa çoğunluğun zinciri doğru kabul ediliyor. Farklı olan düğüm tespit edilip çoğunluğun kopyasıyla onarılıyor.
- **Gizli oy ve makbuz:** Deftere oyun kendisi yazılmıyor; onun yerine `taahhüt = SHA-256(oylama | seçim | makbuz)` değeri yazılıyor. Makbuz kodu yalnızca oy verene ve yalnızca bir kez gösteriliyor. Oy veren kişi, makbuzuyla oyunun deftere doğru kaydedildiğini sonradan kontrol edebiliyor. Makbuzu bilmeyen biri ise özetten seçimi çıkaramıyor.
- **Hız:** İlk sürümde zincir her yazmada baştan doğrulanıyordu. 50.000 bloklu bir defterde bir oy 645 ms, defter sayfası 1,1 saniye sürüyordu. Artık doğrulama sonucu düğümün sürüm numarasıyla birlikte saklanıyor. Düğüm değişmediyse zincir yeniden doğrulanmıyor; değiştiyse doğrulama tekrarlanıyor ve kurcalama yine yakalanıyor (Bölüm 13).

9. Yapay zeka üye

Forumda **Bilge** adında bir yapay zeka üye var. Bilge'nin tek görevi özet yazmak: uzun tartışmaların özetini ve her turun sonucunu yazıyor. Oy vermiyor, fikir önermiyor ve konu açmıyor. Özetler **kural tabanlı** olarak üretiliyor; dışarıdaki bir dil modeli kullanılmıyor. Bu kararın üç nedeni var:

- Üyeler özeti oylamadan önce okuyor. Bir dil modelinin uydurduğu tek bir sayı (halüsinasyon) bile kararı etkileyebilir (S02-43).
- Mesajları dış bir servise göndermek, kişisel verinin dışarı çıkması anlamına geliyor. Ayrıca her çağrı hem ücretli hem de yavaş (S01-34, 35).
- Türkçe bir metin, aynı anlamdaki İngilizce metinden 1,4–1,7 kat daha fazla token tutuyor; maliyet de aynı oranda artıyor (Bölüm 15).

Bu yüzden özetteki her sayı doğrudan veritabanından alınıyor. Oranlar yuvarlanmıyor, aşağı kesiliyor: %74,9 hiçbir zaman "%75" diye yazılmıyor, çünkü %75 ezici üstünlük eşiği ve yuvarlanmış bir oran okuyanı yanıltabilir. Aynı veriden her zaman aynı özet çıkıyor; bunu bir test denetliyor.

10. SOLID ilkeleri ve tasarım desenleri

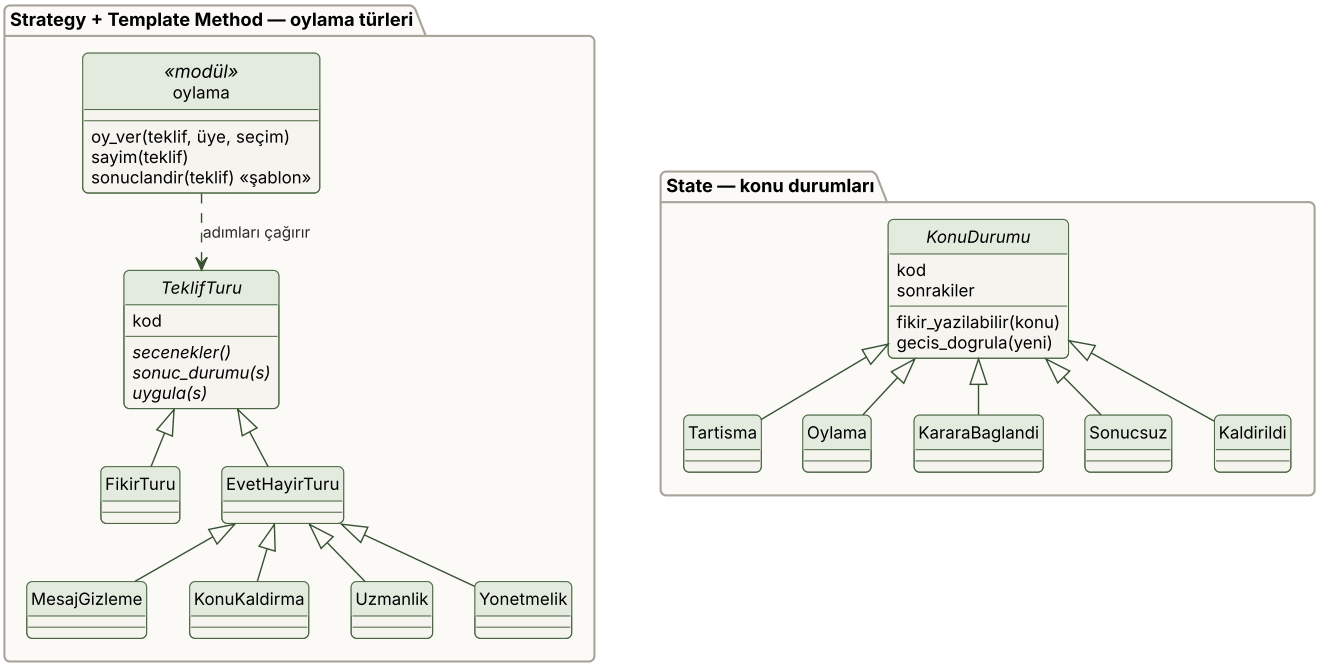
Desenler yalnızca somut bir sorunu çözdükleri yerlerde kullanıldı. Bu kararda ders sunumundaki iki ilke yol gösterdi: desen bir amaç değil, araç olmalı (TD-3) ve gereksiz karmaşıklıktan kaçınılmalı (TD-57). Aşağıdaki tablolarda "önceki sorun" sütunu, düzenlemeden önceki kodda gerçekten bulunan sorunu gösteriyor.

İlke	Önceki sorun	Şimdiki durum
S — Tek sorumluluk	150 satırlık tek bir fonksiyon kimlik, CSRF, zamanlayıcı, hata sayfaları ve şablon filtrelerini birlikte yönetiyordu.	Her iş ayrı bir modülde: `istek`, `hata_sayfalari`, `sablon`, `denetim`, `gorunum`.

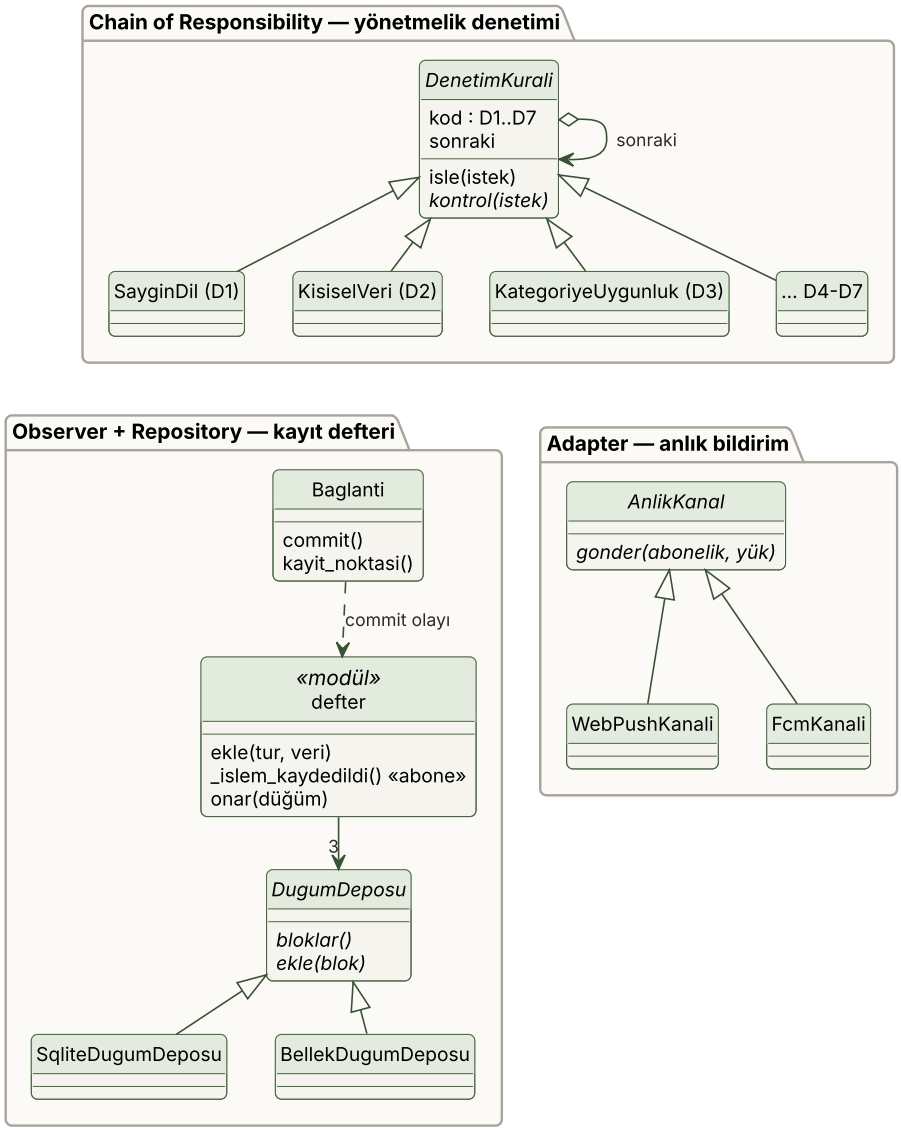
İlke	Önceki sorun	Şimdiki durum
O — Açık/kapalı	Yeni bir oylama türü eklemek için 5 dosyaya yeni bir `if tip == …` dalı yazmak gerekiyordu.	Yeni bir oylama türü için yeni bir sınıf, yeni bir denetim maddesi için yeni bir halka, yeni bir bildirim kanalı için yeni bir adaptör yazmak yetiyor.
L — Liskov	Kapalı bildirim kanalı abone kabul ediyor, ardından her gönderimde hata veriyordu.	Bütün alt türler üst türün yerine sorunsuz kullanılabiliyor. Gerçek ve sahte depoların aynı sonucu verdiğini testler denetliyor.
I — Arayüz ayrımı	Belirgin bir sorun yoktu.	Arayüzler küçük tutuldu: bildirim kanalında 3, denetim maddesinde 1, defter deposunda 5 yöntem var.
D — Bağımlılığın tersine çevrilmesi	Veritabanı modülü (altyapı), üst katmandaki defteri içe aktarıyordu.	Defter, veritabanının commit olayına abone oluyor (Observer); veritabanı defteri tanımıyor.

Desen	Çözdüğü sorun	Agora'daki yeri
State (TD-43)	`durum == …` kontrolleri koda dağılmıştı. Unutulan bir kontrol, kaldırılmış bir konunun geçmişini açık bırakıyordu.	`konu_durumlari.py`: 5 durum sınıfı, izinler ve geçiş tablosu.
Strategy + Template Method (TD-36, 41)	Altı oylama türünün sonuç kuralları uzun bir `if/elif` zincirindeydi.	`teklif_turleri.py`: her tür ayrı bir sınıf. `oylama.sonuclandir` adımları her zaman aynı sırayla çağırıyor.
Chain of Responsibility (TD-44)	D1–D7 maddeleri 77 satırlık tek bir fonksiyonun içindeydi.	`denetim.py`: her madde zincirin bir halkası.
Observer (TD-38)	Altyapı katmanı üst katmanı doğrudan çağırıyordu.	Defter, veritabanının commit olayına abone.
Repository (TD-51)	Düğüm dosyalarına erişim uzlaşma mantığının içine gömülüydü; test için disk gerekiyordu.	`DugumDeposu`: biri SQLite, biri bellek üzerinde iki gerçekleme.
Adapter (TD-24)	Web Push ile Firebase'in arayüzleri birbirinden farklıydı.	`AnlikKanal`: iki kanal tek bir arayüzün arkasında.
Memento (TD-47)	Bir konuda çıkan hata, zamanlayıcının o ana kadar yaptığı bütün işi geri alıyordu.	Her oylama için bir kayıt noktası açılıyor ve kuyrukların o anki hâli saklanıyor. Hata olursa yalnızca o oylama eski hâline dönüyor.
Facade (TD-28)	Konu sayfasının rotası 7 alt sistemi tek tek çağırıyordu.	`gorunum.konu_sayfasi()`: rota yalnızca bu işlevi çağırıyor.

Bilerek kullanılmayan desenler: Builder, Abstract Factory, Visitor ve Mediator kullanılmadı, çünkü projede bu desenlerin çözeceği bir sorun yok. Singleton'a da gerek kalmadı; Python'da bir modül programda zaten tek kopya olarak yükleniyor. Iterator, Command ve Factory gibi desenler Python'da fonksiyon, kapanış (closure) ve sözlükle çok daha sade biçimde yazılabiliyor. Flask'ın kendisi de bazı desenleri hazır olarak kullanıyor: `before_request` zinciri bir Chain of Responsibility, `@route` kaydı ise bir Registry örneği.



Şekil 9. Solda oylama türleri (Strategy + Template Method), sağda konu durumları (State). `sonuclandir()` işlevi adımları hep aynı sırayla çağırıyor: sayım → sonuç durumu → uygulama. Türe özgü adımlar alt sınıflardan geliyor.



Şekil 10. Yönetmelik denetimi (Chain of Responsibility), kayıt defteri (Observer + Repository) ve anlık bildirim (Adapter).

11. Problem çerçeveleme ve ölçüm

"Bir forum yapın" isteğinin arkasında altı ayrı sorun vardı. Bunların üçü yapay zeka gerektirmiyor: söz hakkının eşit dağılmaması, tartışmaların bitmemesi ve oy sayımına güvenilmemesi. Bu üç sorun oylama kurallarıyla ve kayıt defteriyle çözüldü. Diğer üçü yapay zekaya aday: tartışma özeti, içerik denetimi ve aynı konunun tekrar tekrar açılması. Her bileşen için önce "Burada yapay zeka gerçekten gerekli mi?" sorusu soruldu (H2-9):

Bileşen	Görev türü	Karar
Sayım ve eleme	Makine öğrenmesi gerekmiyor	Kural yazıldı. Kural yönetmelikte duruyor ve herkesin anlayabileceği biçimde açıklanabilmesi gerekiyor.
D1 hakaret	İkili sınıflandırma	Etiketli veri olmadığı için şimdilik kural kullanılıyor. Gizleme oylamaları ve şikayetler zamanla etiketli veri olarak birikecek.
D2 kişisel veri	İkili sınıflandırma	Kural yeterli, çünkü telefon, e-posta, T.C. kimlik ve IBAN'ın biçimleri belli.
D3 kategori önerisi	7 sınıflı sınıflandırma	Hatanın bedeli düşük (yalnızca uyarı gösteriliyor), bu yüzden kural yeterli.
Özet	Metin üretimi	Sayılar kuralla üretiliyor. Serbest metin özeti ancak insanın onayladığı bir modelle yazılabilir.

> **Problem tanımı (tek cümle, H2-50).** Konu ya da mesaj yazan üyenin, metninin bir kişiyi aşağılayıp aşağılamadığını (D1) ve kişisel veri içerip içermediğini (D2) göndermeden önce görüp düzeltebilmesi için her metni ikili olarak sınıflandırmak.
>
> **Başarı ölçütü:** Kurallar dondurulduktan sonra yazılmış ve daha önce hiç görülmemiş bir örnek kümesinde D1'in kesinliği ve F1 puanı en az 0,80, D2'nin duyarlılığı en az 0,80 olmalı. Bir yapay zeka modeli, kuralı F1'de en az 0,10 geçmedikçe kuralın yerine kullanılmayacak.

11.1 Metrikler

Başarı dört düzeyde ölçülüyor. Son düzeydeki koruyucu metrikler, bir iyileştirme yapılırken bozulmaması gereken ölçüler.

- **İş:** konuların karara bağlanma oranı ve itiraz oranı.
- **Ürün:** karara kadar geçen süre, katılım oranı ve fikir başına yanıt sayısı.
- **Model:** D1 için kesinlik, çünkü masum bir mesajı engellemek pahalı. D2 için duyarlılık, çünkü kişisel verinin gözden kaçması pahalı. D3 için makro F1.
- **Koruyucu:** oy gücü eşitsizliği (Gini katsayısı), defter tutarsızlığı (0 olmalı), özetteki sayıların doğruluğu (%100 olmalı) ve yanıt süresi (p95 < 200 ms).

11.2 Temel çizgi ve sonuçlar

Ölçümde toplam 166 örnek kullanıldı: 85'i geliştirme, 43'ü test, 38'i görülmemiş son küme için. Kurallar önce iki basit yöntemle karşılaştırıldı: her mesaja "temiz" demek (çoğunluk sınıfı) ve rastgele tahmin (1000

tekrarın ortalaması). Son küme, kurallar dondurulduktan sonra yazıldı ve ölçümden önce kayda geçirildi. Böylece kurallar bu kümeye göre ayarlanamadı.

Madde · metrik	Çoğunluk	Rastgele	Kural · test	Kural · son küme	Ölçüt
D1 hakaret · kesinlik / F1	0 / 0	0,53 / 0,50	0,83 / 0,83	0,71 / 0,71	**sağlanmadı**
D2 kişisel veri · duyarlılık	0	0,49	0,80	0,83	**sağlandı**
D3 kategori · makro F1	0,04	0,13	0,87	0,49	**sağlanmadı**

Son kümedeki düşüş, test kümesindeki sonuçların iyimser olduğunu gösterdi. Son kümedeki hatalardan bazıları ve nedenleri:

Hata örneği (son küme)	Neden
"*Aptal telefonlar* dönemine dönmek ister misiniz?" cümlesini D1 yanlışlıkla engelledi.	Kelime bir kişiyi değil, bir nesneyi niteliyor. Kelime listesi bağlamı anlayamıyor.
"Kafana taş mı düştü?" cümlesini D1 kaçırdı.	Hakaret bir deyimle yapılmış; listede eşleşen bir kelime yok.
"Moda Caddesi No 15 Daire 7..." adresini D2 kaçırdı.	Serbest biçimde yazılmış adres hiçbir kalıba uymuyor.
"Yurtlarda grip aşısı kampanyası" için D3, Sağlık yerine Eğitim kategorisini önerdi.	Yalnızca "yurt" kelimesi eşleşti; "grip" ve "aşı" kavram listesinde yoktu.

Karar. Ölçüt önceden yazıldığı için sonuçlara bakıp hedef değiştirilmedi (H2-43). D2 ölçütü sağladı ve engellemeye devam ediyor. D1'in engellediği mesajların ise %29'u aslında masumdu. Masum bir mesajı engellemenin maliyeti 5 birim, bir hakaretin gözden kaçmasının maliyeti 1 birim sayıldığında (H2-31), engelleyen kuralın toplam maliyeti 12 birim, hiçbir şeyi engellememenin maliyeti ise 7 birim tutuyor. Bu yüzden D1 artık **yalnızca uyarıyor**: metin yayımlanıyor, yazana bir uyarı gösteriliyor, gözden kaçan hakareti de topluluk gizleme oylamasıyla kaldırabiliyor. Topluluk isterse D1'i oylamayla yeniden "Engeller" düzeyine çıkarabilir. Sonraki adım gerçek veri toplamak: gizleme oylamaları ve şikayetler biriktikçe, kuralın yerine geçebilecek bir modeli eğitmek için etiketli veri oluşacak.

Not: Ölçüm örneklerini de kuralları da aynı YZ aracı yazdığı için sayılar iyimser olabilir. Ön otopsi (pre-mortem, H2-47) çalışmasında en büyük risk olarak *katılımsızlık* çıktı: kimse oy vermezse yeter sayıya ulaşılamıyor ve konular sonuçsuz kalıyor.

12. Güvenlik ve kişisel veri

- **Şifre:** Şifreler scrypt ile özetlenerek saklanıyor; şifre kurtarma kodu da yalnızca özet olarak tutuluyor. Üst üste 5 hatalı girişten sonra hesap 10 dakika kilitleniyor. Şifre değiştirildiğinde eski oturumlar ve API anahtarları geçersiz oluyor.
- **Web saldırıları:** Her formda bir CSRF anahtarı var; bu anahtar, başka bir sitenin üye adına form göndermesini engelliyor. İçerik güvenlik politikası (CSP) ve diğer güvenlik başlıkları her yanıtla gönderiliyor. SQL sorguları parametreli yazıldı, şablonlar da otomatik kaçış kullanıyor; bu iki önlem SQL enjeksiyonunu ve XSS saldırılarını önlüyor. Girişten sonra yalnızca site içindeki bir adrese yönlendirme yapılıyor. Bildirim adresleri yalnızca bilinen servislere gidebiliyor (SSRF engeli). Büyük metinler için de bir boyut sınırı var.

- **Ayrı yönetici girişi:** Yönetim panelinin kendi giriş sayfası ve üye oturumundan bağımsız bir oturumu var. Üye olarak giriş yapmak paneli açmıyor. Biri çıkış yaptığında diğer oturum etkilenmiyor.
- **Kişisel veri (KVKK):** Ad, adres ve doğum tarihi hiçbir sayfada ve API yanıtında görünmüyor. Deftere kişisel veri değil, yalnızca özetler yazılıyor. Üye ağında 3 kişiden küçük görüş grupları gösterilmiyor, çünkü iki kişilik bir grup o iki kişinin aynı oyu verdiğini ortaya çıkarırdı.

13. Test, kalite ve başarım

- **204 otomatik test** yaklaşık 15 saniyede tamamlanıyor. Testler iş kurallarını, web sayfalarını, hata düzeltmelerini, desenleri ve ölçümü kapsıyor. Hepsini çalıştırmak için `python -m pytest` komutu yeterli.
- **Sürekli entegrasyon:** GitHub Actions her gönderimde testleri, statik denetimi (pyflakes) ve denetim ölçümünü çalıştırıyor. Testlerden biri bile başarısız olursa site yayımlanmıyor.
- **Önce test:** Hata düzeltmeleri için yazılan testlerin, düzeltmeden önceki kodda gerçekten başarısız olduğu git geçmişi üzerinden denetlendi. Böylece testlerin hatayı gerçekten yakaladığı kanıtlandı.
- **Sahte nesneler:** Defter testleri diske dokunmadan bellekteki sahte bir depoyla, bildirim testleri sahte bir kanalla, zamana bağlı testler de ileri sarılabilen bir saatle çalışıyor. Bu sayede testlerin toplam süresi yaklaşık 60 saniyeden 15 saniyeye indi.

Başarım için sunucunun yanıt süreleri, düzenlemelerden önce ve sonra aynı koşullarda ölçüldü:

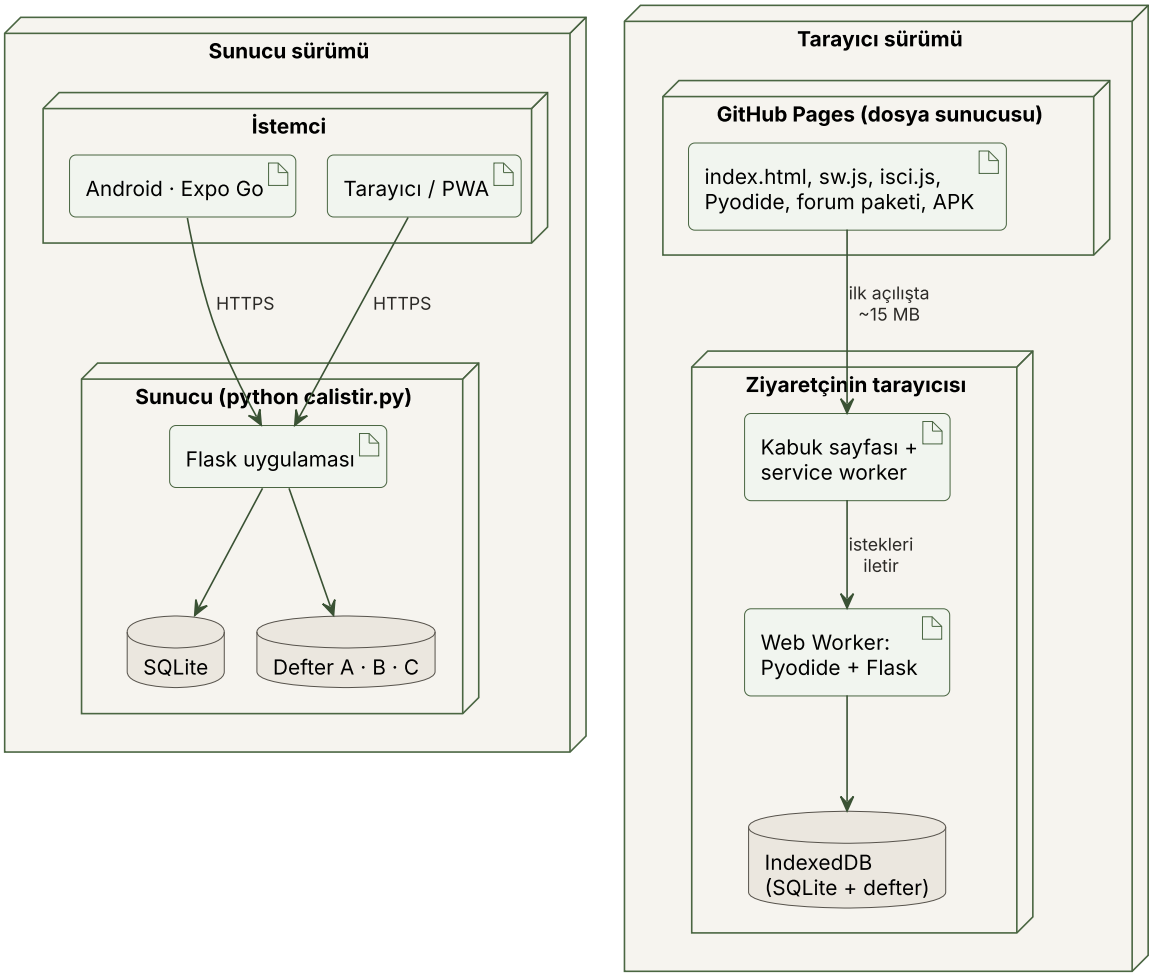
Sunucu tarafı yanıt süresi (ms)	Önce	Sonra
Oy verme, demo verisi (p95)	18,6	10,1
Oy verme, defterde +50.000 blok (p95)	645,2	9,5
Kayıt defteri sayfası, +50.000 blok (p95)	1.145,5	11,7
Konu akışı, 1.019 konu (p50)	77,0	48,7

Hedef: p95 < 200 ms. p95, isteklerin %95'inin bu süreden kısa sürede yanıtlandığı anlamına geliyor; p50 ise ortanca süre. Ölçüm komutu: `python olcum/gecikme_olcumu.py`.

14. Dağıtım ve mobil

Agora iki biçimde çalışabiliyor: bir sunucuda ya da doğrudan ziyaretçinin tarayıcısında. İki sürüm de aynı Python kodunu kullanıyor (Şekil 11).

- **Sunucu sürümü:** `python calistir.py` komutuyla (Windows'ta `baslat.bat` ile) başlatılıyor. Bütün kullanıcılar aynı veritabanını paylaşıyor.
- **Tarayıcı sürümü (GitHub Pages):** GitHub Pages yalnızca dosya sunabiliyor, Python çalıştıramıyor. Bu yüzden aynı Python kodu ziyaretçinin tarayıcısında Pyodide ile çalışıyor. Service worker, sayfanın her isteğini yakalayıp tarayıcıdaki Flask uygulamasına iletiyor. Veriler tarayıcının IndexedDB deposunda saklanıyor. İlk açılışta yaklaşık 15 MB indiriliyor; sonraki açılışlarda site internet olmadan da açılabiliyor. Her ziyaretçinin kendine ait bir demo kopyası oluyor.
- **Mobil:** Telefonda üç seçenek var: siteyi "Ana ekrana ekle" ile uygulama gibi kurmak (Android ve iPhone), Android uygulamasını (APK) indirmek ya da Expo Go ile açmak. APK, Capacitor ile hazırlandı ve siteyi tam ekran açıyor. Her yayında GitHub Actions tarafından yeniden derleniyor.

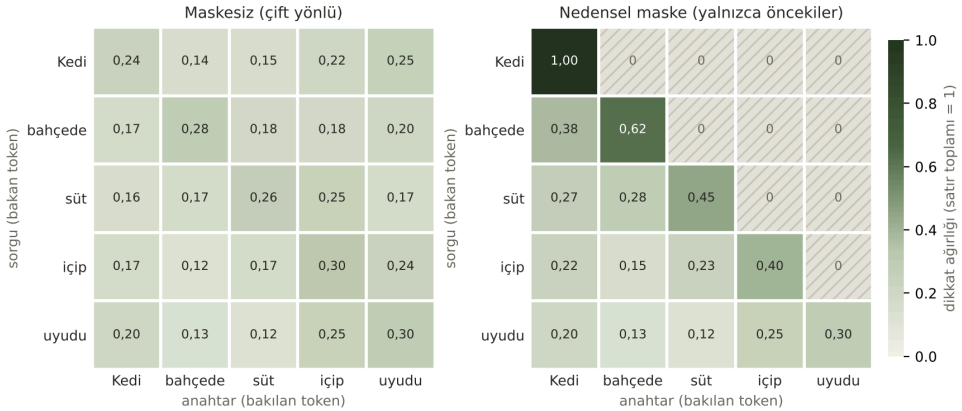


Şekil 11. Dağıtım diyagramı. Solda sunucu sürümü, sağda GitHub Pages üzerinden açılan tarayıcı sürümü yer alıyor.

15. Mini laboratuvar (S02-50, S01-49)

Ders sunumlarındaki laboratuvar görevleri `laboratuvar/` klasöründe yapıldı. Sonuçlar aşağıda:

Deney	Sonuç
Türkçe ve İngilizce token sayısı	10 Türkçe cümle, İngilizce çevirilerinden ortalama 1,69 kat (cl100k) ve 1,36 kat (o200k) daha fazla token tutuyor. Tek başına "Avrupalılaştıramadıklarımızdan" kelimesi 14 ve 12 token. Token sayısı arttıkça maliyet, gecikme ve bağlam penceresi kullanımı da aynı oranda artıyor.
Sıcaklık	Güncel Claude modelleri sıcaklık parametresini kabul etmediği için deney, yerel bir kelime ikilisi (bigram) modelinde yapıldı. 10 denemede T = 0 / 0,7 / 1,5 için sırasıyla 1 / 9 / 10 farklı yanıt çıktı. Yani sıcaklık arttıkça yanıtlar çeşitleniyor.
Dikkat ısı haritası	Slayttaki NumPy dikkat kodu yeniden üretildi; sonuç Şekil 12'de.
YZ'nin önerdiği import'lar	Önerilen 7 paketin ve 8 import satırının hepsi gerçekten var. Ancak kod çalıştırılınca iki gizli gereksinim ve bir anlam kayması ortaya çıktı.
YZ'nin yazdığı testler	T.C. kimlik denetimi için yazılan 16 testin 16'sı da geçti. Ama mutasyon testinde koda bilerek sokulan 10 hatadan biri yakalanmadı, çünkü "10. hane hatalı" testi 11. haneyi de bozuyordu. Buradan çıkan ders: testlerin geçmesi, kodun doğru olduğunu kanıtlamıyor.



Şekil 12. Nedensel maskeli dikkat ağırlıkları. Köşegenin üstü 0, çünkü bir token kendinden sonra gelen tokenları göremiyor. Her satırın toplamı 1.

16. Sınırlar ve YZ kullanım beyanı

Projenin bilinen sınırları şunlar:

- D1 ve D3 kural tabanlı birer temel çizgi; ikisi de başarı ölçütünü sağlamıyor.
- Üyelerin gerçek kimliği doğrulanmıyor; sahte hesaplara karşı yalnızca hız sınırı var.
- SQLite saniyede yaklaşık 100 yazma işlemiyle sınırlı. Büyük bir topluluk için PostgreSQL gibi bir sunucu veritabanı gerekir.
- Yedek dosyası şifrelenmiyor.
- Tarayıcı sürümü kişisel bir demo; veriler kullanıcılar arasında paylaşılmıyor.

Bu nedenlerle Agora, gerçek kullanıma açılmış bir ürün değil, bir **prototip**.

> **YZ kullanım beyanı (S01-46, 47; H2-59).** Forumun ilk sürümü benim commit'im (1 Ekim 2026). Sonraki commit'lerde Claude Code (Anthropic) kodlama aracını kullandım. Araç şu işlerde kullanıldı: analiz ve tasarım belgeleri, SOLID ve desen düzenlemeleri, hata düzeltmeleri ve testleri, ölçüm betikleri ve örnekleri, tarayıcı sürümü, mini laboratuvar, diyagramlar ve bu rapor. Ürün fikri, ilk sürüm, hedefler, ders kaynakları, kapsam kararları, gözden geçirme ve teslim bana ait. Testleri de YZ yazdığı için düzeltme testlerinin eski kodda başarısız olduğu ayrıca denetlendi; başarı ölçütleri de sonuçlar görülmeden önce yazıldı.

Kaynaklar: Ders sunumları (S01, S02, H2, TD); Chip Huyen, *Designing Machine Learning Systems* (2022) ve *AI Engineering* (2025); Gamma vd., *Design Patterns* (1994). Ayrıntılı belgeler: `docs/analiz.md`, `docs/tasarim.md`, `docs/rehber.md`. Bütün diyagramların kaynakları `docs/rapor/sekiller/` klasöründe.